

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓名：张序鸿

学院：竺可桢学院

专业：计算机科学与技术

学号：3230103265

指导教师：杨莹春

2024 年 10 月 12 日

浙江大学实验报告

课程名称： 计算机组成与设计 实验类型： 综合

实验项目名称： 复杂操作实现——mul、div、floatadd

学生姓名： 张序鸿 专业： 混合班 学号： 3230103265

指导老师： 杨莹春、潘之杰 助教： 张立冬、马致远

实验地点： 东 4-509 实验日期： 2024 年 10 月 9 日

一、实验目的和要求

1. 复习二进制加减、乘除的基本法则
2. 掌握补码的基本原理和作用
3. 了解浮点数的表示方法及加法运算法则
4. 进一步了解计算机系统的复杂运算操作

二、实验内容和原理

内容：

- 任务一：设计实现乘法器
- 任务二：设计实现除法器
- 任务三：设计实现浮点加法器

原理：

2.1 设计实现乘法器

2.1.1 乘法的基本概念

计算机实现乘法器的原理基于模拟手工运算的执行过程（竖式乘法）：

每个中间结果产生后直接
与当前的乘积累加

每产生一个中间结果被
乘数向左移动一位

1000

x 1001

1000

0000

0000

1000

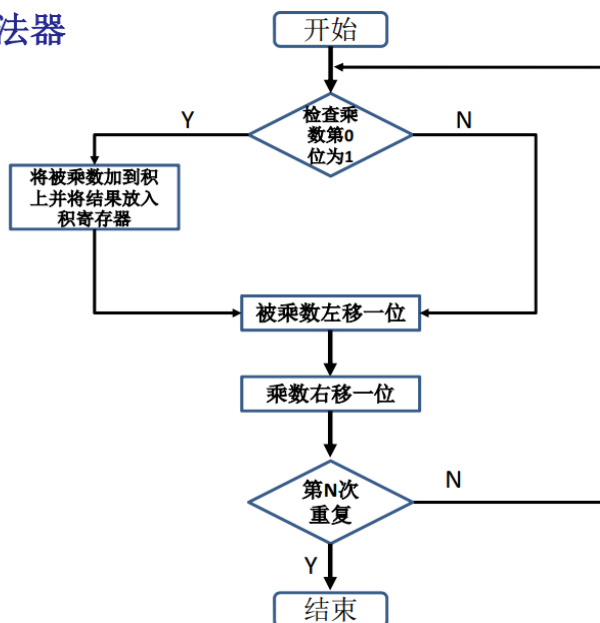
1001000

N 位乘法计算过程 ($X * Y$) 主要有两个判定过程:

(1) 判断 Y 的最低位是 0 还是 1, 如果是 1, 则需要把 X 加到部分积上, 若为 0, 则需要把 0 加到部分积 $\rightarrow$ X 左移一位, 再让 Y 右移一位。

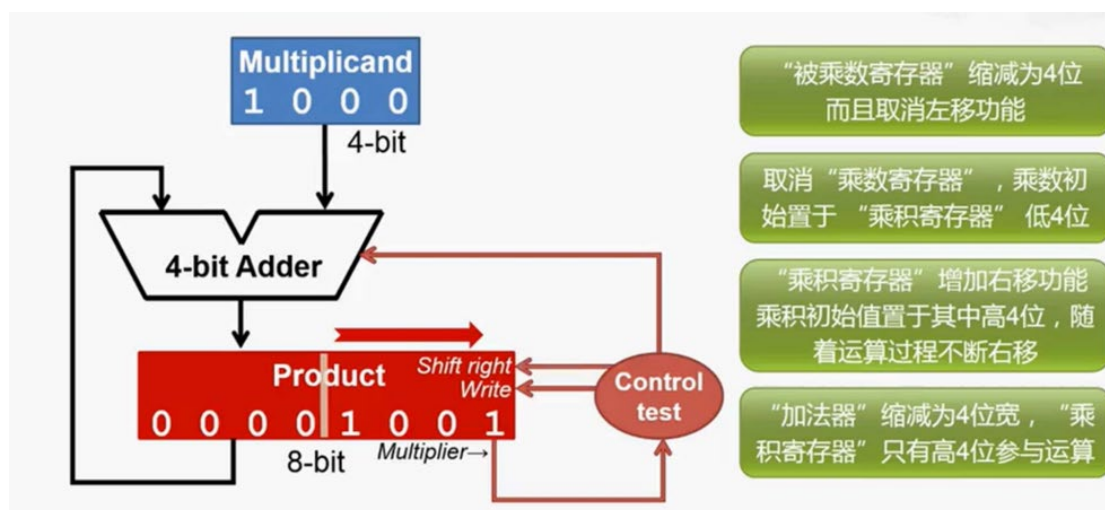
(2) 判断 Y 是否为 0 (移位次数是否达到 N), 循环结束或继续循环。

N位乘法器



2.1.2 乘法的优化实现 (减少了不必要的硬件)

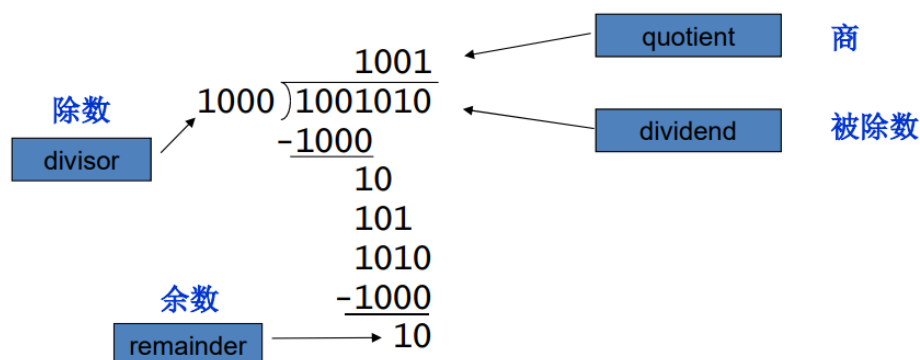
书中给出了 3 种版本, 此处只给出了 version 3 的最终优化版 (改图为 4bit 乘法, 实际实现则是 N bit 乘法):



2.2 设计实现除法器

2.2.1 除法的基本概念

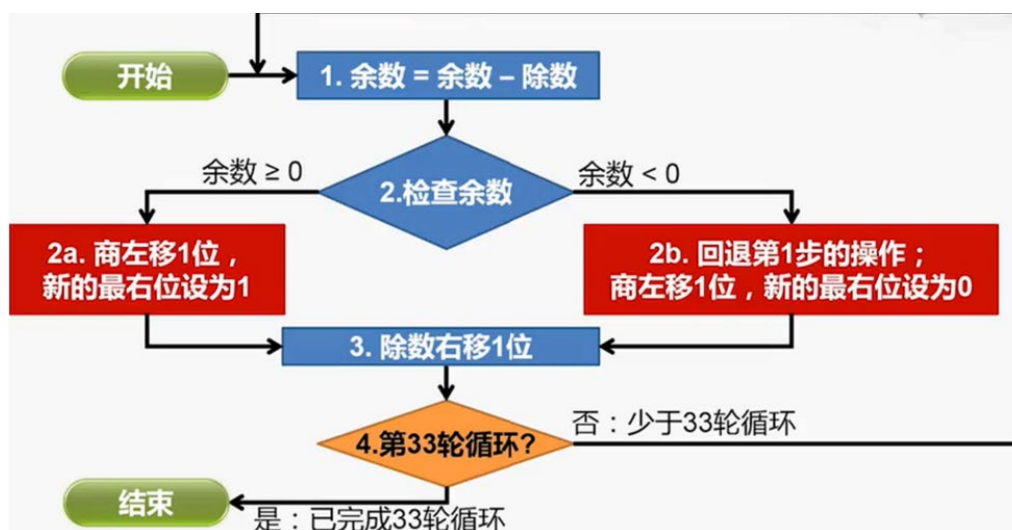
计算机实现除法器的原理也是基于模拟手工运算的执行过程：



N 位除法计算过程 ($X \div Y$) 主要有两个判定过程：

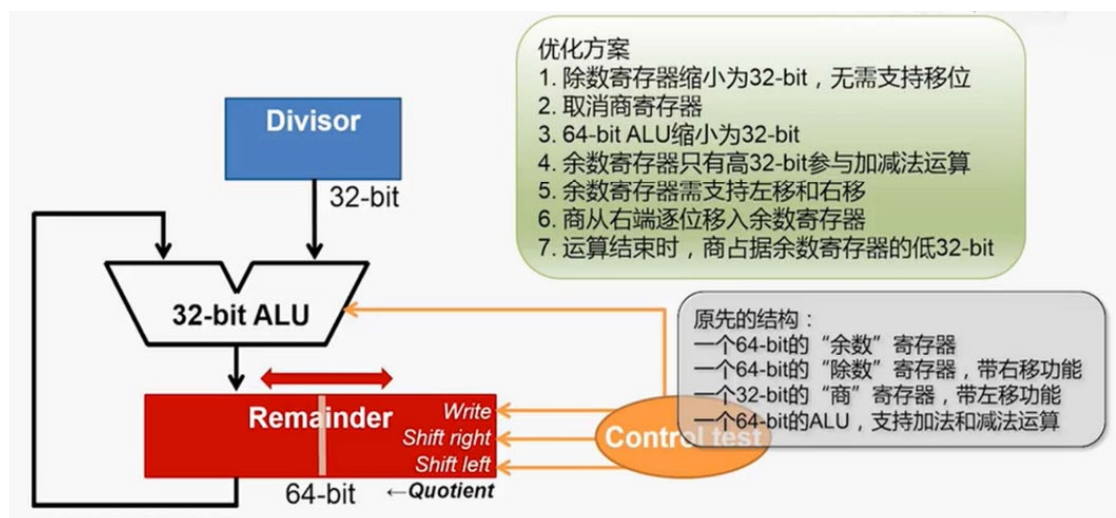
(1) 预先计算当前余数, 并根据余数的正负进行商寄存器的下一步操作, 主要为末尾置 0 余数左移、末尾置 1 余数更新。

(2) 判断移位次数是否达到 N, 循环结束或继续循环。



2.2.2 除法优化实现

除法硬件的优化, 最主要体现在余数寄存器和商寄存器的合并, 并且仅让余数寄存器的高位参与加减, 使得 ALU 的位数得以减半。



2.3 设计实现浮点加法器

2.3.1 浮点数的基本概念

• 浮点数的表示：

机器浮点数由阶码、尾数以及符号位来组成：

计算机中一个任意进制数 N 可以写成

$$N = R^e \times m = 2^E \times M = 2^{\pm e} \times (\pm m)$$

m : **尾数**，是一个纯小数。

e : 浮点的**指数**，是一个整数。

R : **基数**，对于二进计数值的机器是一个常数，一般规定 R 为2

• 浮点数规格化：

就像十进制中的科学计数法，对于二进制的非0浮点数，要求尾数的绝对值应当 ≥ 0.5 ，即尾数 m 的最高有效位是1，对于不符合这类要求的数应通过 尾数移位 + 阶码修改 来实现规格化处理。

2.3.2 浮点数标准格式 IEEE754

IEEE754 标准规定：尾数 采用 原码，阶码 采用“移码”，基为2。

IEEE754 标准浮点数包括三个部分 S（尾数符号）、M（尾数）、E（阶码）。

常用的有单精度浮点数（32 bit）和 双精度浮点数（64 bit），



- 这里采用了隐藏位技术, 根据规格化技术, 尾数最高位为 1, 因此可以将尾数左移并默认 1 的存在, 使得 M 可以多存一位二进制数, 提高精度。

- “移码”: 为避免指数正负引入新的符号位, 阶码会进行偏移(单精度浮点数偏移值为 127), 如 -2 对应的 E 为 $-2 + 127 = 125 = 011111101$ 。

一个规格化的32位浮点数 x 的真值为:

$$x = (-1)^s \times (1.M) \times 2^{E-127} \quad e = E - 127$$

一个规格化的64位浮点数 x 的真值为:

$$x = (-1)^s \times (1.M) \times 2^{E-1023} \quad \text{这里 } e \text{ 是真值, } E \text{ 是机器数}$$

2.3.3 浮点数的加减法

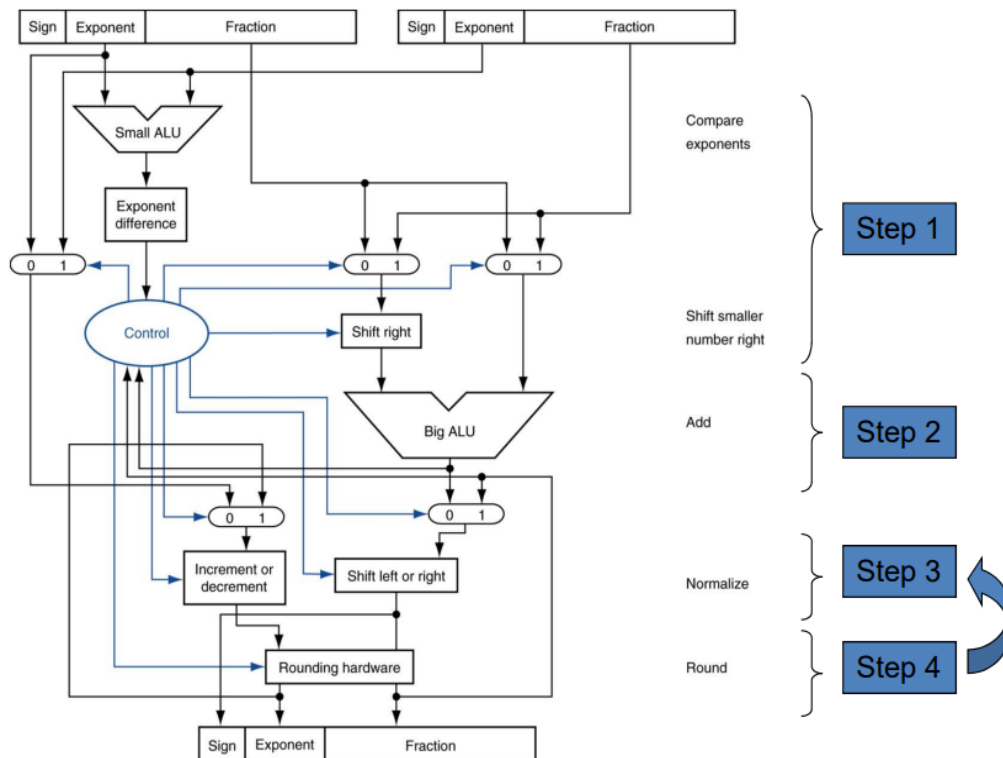
浮点数的加减法, 主要操作是将阶数不同的两个数进行尾数移位, 从而能够进行简单的 add 操作。

完成浮点加减运算的操作过程大体分为:

- (1) 0 操作数的检查;
- (2) 比较阶码大小并完成对阶;
- (3) 尾数进行加或减运算;
- (4) 结果规格化。
- (5) 舍入处理。
- (6) 溢出处理。

其中, 舍入操作需要根据移位后的多余低位尾数来判断, 常用简单的舍弃最高位“0 舍 1 进”的方法。

2.3.4 浮点数的加法硬件实现



三、实验过程和数据记录

3.1 设计实现乘法器：

(1) Vivado 新建工程

- 利用 Vivado 新建工程，命名 mul32 ；
- 新建文件 mul32.v ；
- 最后正确选择 FPGA 芯片——xc7a100tcsg324-1

(2) mul32 具体实现：

- 数据接口如下：

```
module mul32(  
    input clk,  
    input rst,  
    input [31:0] multiplicand,  
    input [31:0] multiplier,  
    input start,  
    output reg [63:0] product,  
    output finish  
);
```

- 逻辑实现：

```
integer cnt = 0;
reg fin;
assign finish = fin;

always @(posedge clk or posedge start or rst) begin
    if(rst) begin
        product <= 64'h0;
        fin <= 1'b0;
    end
    else if(start) begin
        product[63:0] <= {32'b0,multiplier};
        cnt <= 0;
        fin = 1'b0;
    end
    else begin
        if(cnt<32) begin
            if(multiplier[cnt] == 1'b1)
                product<={ (product[63:32]+multiplicand),product[31:0]} >> 1;
            else
                product <= product >> 1;
                cnt <= cnt+1;
                fin <= 1'b0;
            end
        else
            fin <= 1'b1;
        end
    end
end
```

(3) mul32 模块行为仿真：

TB 仿真采用固定输入，从而便于检查结果的正确性：

```
module mul32_tb();
    reg clk;
    reg rst;
    reg[31:0] multiplicand;
    reg[31:0] multiplier;
    reg start;
    wire[63:0] product;
    wire finish;
    initial begin

        clk = 0;
```



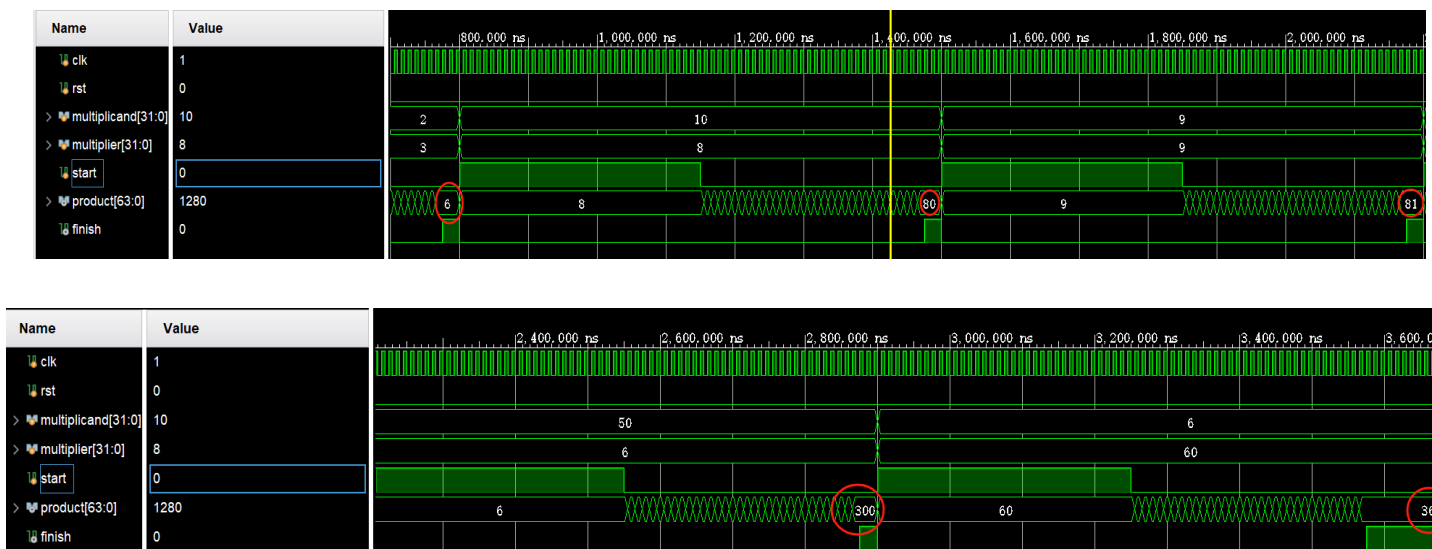
```

rst = 1;
multiplicand = 0;
multiplier = 0;
start = 0;
#100
rst = 0;
start = 1;
multiplicand = 32'd2;
multiplier = 32'd3;
#350
start = 0;
#350
start = 1;
multiplicand = 32'd10;
multiplier = 32'd8;
#350
start = 0;
#350
start = 1;
multiplicand = 32'd9;
multiplier = 32'd9;
#350
start = 0;
#350
start = 1;
multiplicand = 32'd50;
multiplier = 32'd6;
#350
start = 0;
#350
start = 1;
multiplicand = 32'd6;
multiplier = 32'd60;
#350
start = 0;
#4000 $finish();
end

always #5 clk = ~clk;

mul32 mul32_u(
    clk,rst,multiplicand,multiplier,start,product,finish
);
endmodule

```



利用小学二年级就会的乘法口诀，可以看出 product 的结果与输入数据匹配。

3.2 设计实现除法器：

（1）Vivado 新建工程

- 利用 Vivado 新建工程，命名 div32 ；
- 新建文件 div32.v ；
- 最后正确选择 FPGA 芯片——xc7a100tcsg324-1

（2）mul32 具体实现：

- 数据接口如下：

```
module div32(
    input  clk,
    input  rst,
    input  start,
    input[31:0] dividend,
    input[31:0] divisor,
    output [31:0] quotient,
    output [31:0] remainder,
    output reg finish
);
```

- 逻辑实现：

```

reg[63:0] rem; // 64 位的 余数/商寄存器
reg[31:0] div;
reg [5:0] cnt = 6'b0;
assign quotient = rem[31:0];
assign remainder = rem[63:32];

always@(posedge clk or posedge rst or posedge start) begin
    if(rst || !start) begin
        rem <= 64'b0;
        div <= 32'b0;
        finish <= 1'b0;
        cnt <= 0;
    end
    else if(start && !finish) begin
        if(cnt == 0) begin
            rem <= {32'b0 , dividend};
            div <= divisor;
            finish <= 1'b0;
            cnt <= 1;
        end
        else if(cnt <= 33) begin
            if(rem[63:32] < div) begin
                rem <= rem << 1;
            end
            else if(rem[63:32] >= div) begin
                rem <= ({(rem[63:32] - div), rem[31:0]} << 1) + 1'b1;
            end
            cnt <= cnt + 1;
        end
        else begin
            rem[63:32] <= rem[63:32] >> 1;
            finish <= 1'b1;
        end
    end
end
end

```

- 相较于 mul32 的设计，div32 主要由 start 信号控制（多次除法不用 rst）
- $rem \leq ((rem[63:32] - div), rem[31:0]) \ll 1 + 1'b1;$
 这步操作包括 —— 高 32 位余数的加减、余数寄存器左移、商末尾+1.
- $rem[63:32] \leq rem[63:32] \gg 1;$
 最后一个看似多余的操作，主要由上面这一步造成，因为正确操作顺序

应该是高 32 位余数的加减、商寄存器末尾+1、余数寄存器左移，如下：

```
rem <= (({(rem[63:32] - div),rem[31:0]}) + 1'b1) << 1;
```

但是，我的实现为了减少寄存器的使用（预先计算当前余数的寄存器），在整体逻辑中余数的判断会比理想情况慢一个周期，最终余数会多移一位。因此，最后需要对其进行更正（重写逻辑太麻烦，导致需要更麻烦的更正。。。）

（3）div32 模块行为仿真：

TB 仿真采用固定输入，从而便于检查结果的正确性

```
module div32_tb();
    reg clk;
    reg rst;
    reg [31:0] dividend;
    reg [31:0] divisor;
    reg start;

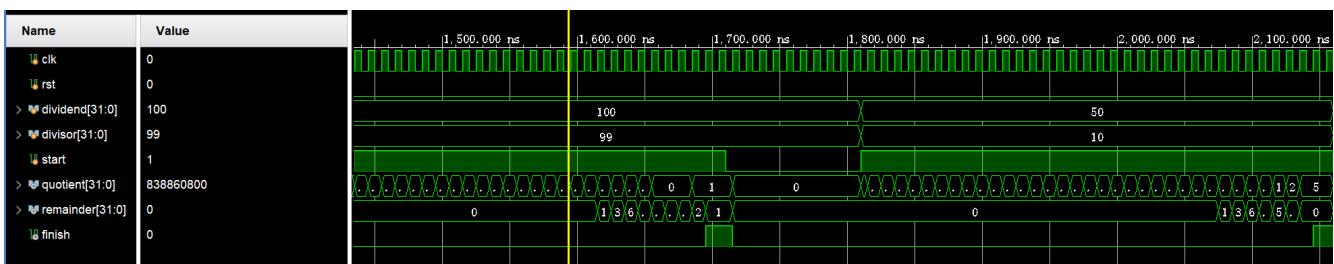
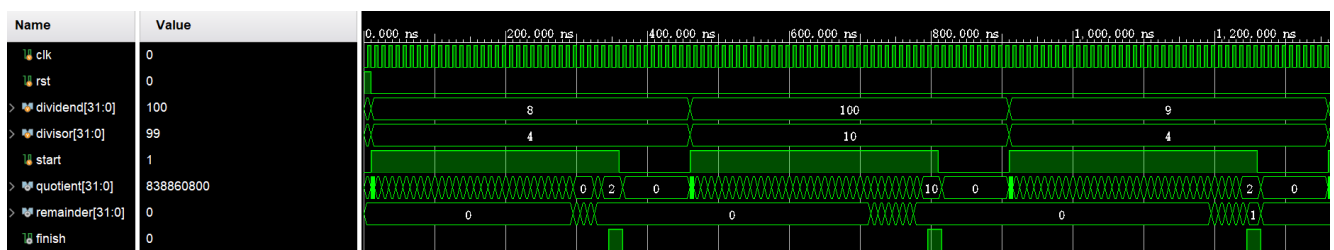
    wire [31:0] quotient;
    wire [31:0] remainder;
    wire finish;
    div32 u_div(
        .clk(clk),
        .rst(rst),
        .dividend(dividend),
        .divisor(divisor),
        .start(start),
        .quotient(quotient),
        .remainder(remainder),
        .finish(finish)
    );

    always #5 clk = ~clk;
    initial begin
        clk = 0;
        rst = 1;
        start = 0;
        dividend = 32'd0;
        divisor = 32'd0;
        #10
        rst = 0;
        start = 1;
        dividend = 32'd8;
        divisor = 32'd4;
```

```

#350
start = 0;
#100
start = 1;
dividend = 32'd100;
divisor = 32'd10;
#350
start = 0;
#100
start = 1;
dividend = 32'd9;
divisor = 32'd4;
#350
start = 0;
#100
start = 1;
dividend = 32'd100;
divisor = 32'd99;
#350
start = 0;
#100
start = 1;
dividend = 32'd50;
divisor = 32'd10;
#350 $stop();
end
endmodule

```



根据仿真结果，当 finish=1 时，每一次的商和余数都相当正确！

3.3 设计实现浮点加法器：

(1) Vivado 新建工程

- 利用 Vivado 新建工程，命名 floatadd ；
- 新建文件 floatadd.v ；
- 最后正确选择 FPGA 芯片——xc7a100tcs9364-1

(2) mul32 具体实现：

- 数据接口 && 变量如下：

```
module floatadd(  
    input clk,  
    input rst,  
    input en,  
    input[31:0] A,  
    input[31:0] B,  
    output [31:0] result,  
    output reg fin  
);  
reg signa,signb,sign,sign_compare; // 符号位 & 符号位比较  
reg[7:0]expa,expb; // 指数部分  
reg[22:0]fracb; // 尾数部分  
reg[7:0]exp,exp_dis; // 结果指数 & 对阶时的阶数差  
reg[24:0]fra_max,fra_min,fra_round; //大小尾数（还原隐藏1，并防止溢出）  
reg[24:0]frac; // 结果尾数（多2位是为了防止进位超出范围）  
reg[31:0]res; // 最终结果  
reg[2:0]state; // 目前状态（采用状态机实现）  
  
wire round;  
assign result = res;  
assign round = fra_round[0];
```

- 逻辑实现：

```
initial begin  
    sign = 0;  
    frac = 0;  
    exp = 0;  
end  
always @(posedge clk) begin  
    case(state)  
        .....  
    endcase  
end
```

I . State = 000 --初始化:

```
3'b000: begin
    fin <= 0;
    signa <= A[31];
    signb <= B[31];
    expa <= A[30:23];
    expb <= B[30:23];
    fracb <= A[22:0];
    fracb <= B[22:0];
    state <= 3'b001;
end
```

II.State = 001 -- 0 操作数的检查:

```
3'b001: begin
    if(A == 0) begin
        sign <= signb;
        exp <= expb;
        frac = fracb;
        state <= 3'b101; // 结束计算
    end
    else if(B == 0) begin
        sign <= signa;
        exp <= expa;
        frac = fracb;
        state <= 3'b101; // 结束计算
    end
    else
        state <= 3'b010;
end
```

III.State = 010 – 比较阶码大小 & 一系列操作

```
3'b010:begin
    if(signa != signb)    sign_compare <= 0;
    else    sign_compare <= 1;    // 判断计算尾数的加减法

    if(expa > expb) begin
        exp <= expa;
        sign <= signa;
        exp_dis <= expa-expb;
        fra_max <= {2'b01, fracb};
        fra_min <= {2'b01, fracb};
    end
```

```

else if(expa < expb) begin
    exp <= expb;
    sign <= signb;
    exp_dis <= expb-expa;
    fra_max <= {2'b01, fracb};
    fra_min <= {2'b01, fracb};
end

else begin
    exp <= expa;
    sign <= signa;
    exp_dis <= 0;
    if(fracb > fracb) begin
        fra_max <= {2'b01, fracb};
        fra_min <= {2'b01, fracb};
    end
    else begin
        fra_max <= {2'b01, fracb};
        fra_min <= {2'b01, fracb};
    end
end
state <= 3'b110;
end

```

IV.State = 011 – 对阶 & 寻找舍入位:

```

3'b011: begin
    fra_min <= fra_min >> exp_dis; // 对阶
    fra_round <= fra_min >> (exp_dis-1); // 获得舍弃的最高位
    state <= 3'b100;
end

```

V.State = 100 – 尾数运算:

```

3'b100: begin
    if(sign_compare)
        frac <= fra_max + fra_min + round;
    else
        frac <= fra_max - fra_min - round;
    state <= 3'b101;
end

```


VI.State = 101 – 结果规格化：

```
3'b101: begin
    if(frac[24] == 1'b1) begin
        exp <= exp + 1;
        frac <= frac >> 1;
    end
    else if(frac[24:23] == 2'b00) begin
        while(frac[24:23] == 2'b00) begin
            exp <= exp-1;
            frac <= frac << 1;
        end
    end
    state <= 3'b110;
end // 确保 frac 的结果为 01.****, 从而方便提取尾数
```

VII.State = 110 – 结束信号输出：

```
3'b110:begin
    fin <= 1;
    if({signa,expa,fraca} != A || {signb,expb,fracb} != B) begin
        state <= 3'b000;
    end
end // 确保每个数据仅计算 1 次，避免 ppt 中的多次计算
```

VIII. Result 赋值 & rst 初始化：

```
always @(posedge fin or posedge rst) begin
    if(en && fin) begin
        if (exp >= 8'b11111111) begin
            res <= {sign, 8'b11111111, 23'b0}; // 无穷大
        end
        else if (exp <= 8'b00000000) begin
            res <= {sign, 8'b00000000, 23'b0}; // 输出零
        end
        else begin
            res <= {sign, exp, frac[22:0]};
        end
    end
    if(rst) begin
        res <= 0;
        sign <= 1'b0;
        exp <= 8'b0;
        frac <= 23'b0;
        fin <= 1'b0;
    end
end
end
```

(3) floatadd 模块行为仿真:

TB 仿真采用固定输入, 从而便于检查结果的正确性

```
module floatadd_tb();
    reg clk;
    reg rst;
    reg [31:0] A;
    reg [31:0] B;
    reg en;

    wire [31:0] result;
    wire fin;

    floatadd add (
        .clk(clk),
        .rst(rst),
        .A(A),
        .B(B),
        .en(en),
        .result(result),
        .fin(fin)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst = 1;
        A = 32'hc000000;
        B = 32'hc000000;
        en = 0;

        #10;
        rst = 1;
        A = 32'hc0a0000; //-5.0
        B = 32'hc090000; //-4.5
        en = 1;          // c1180000 (-9.5)
        rst = 0;

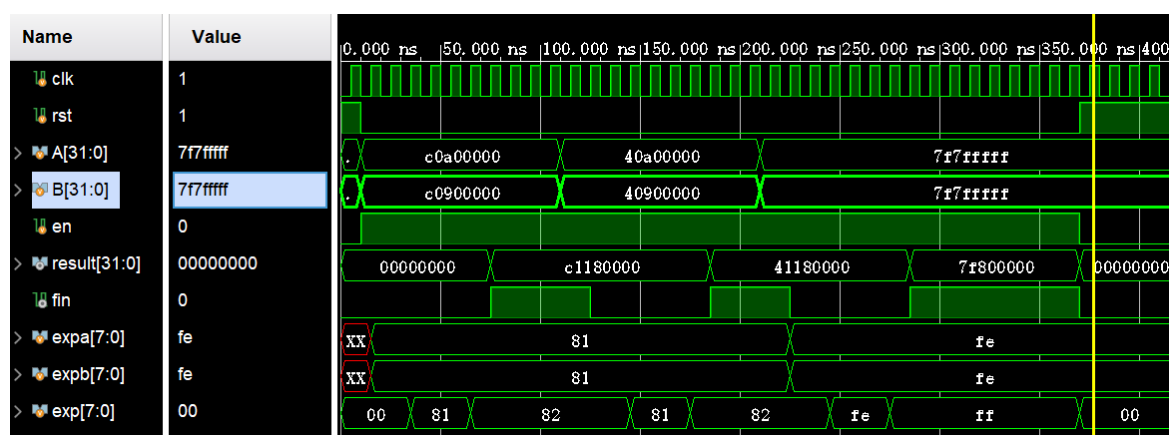
        #100;
        A = 32'h40a0000; //+5.0
        B = 32'h4090000; //+4.5
        en = 1;          // 41180000 (+9.5)
        rst = 0;
    end
endmodule
```

```

#100; // 检查溢出处理
A = 32'h7f7fffff; // 接近最大正数 (3.4028235 × 10^38)
B = 32'h7f7fffff; // 接近最大正数 (3.4028235 × 10^38)
en = 1;
rst = 0;
#160
en = 0;
rst = 1;
$stop();

end
endmodule

```



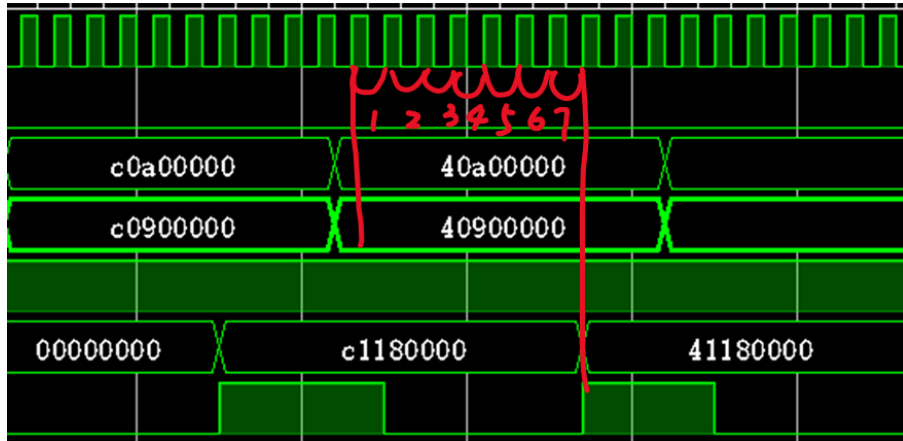
可以看到，前两个基础加法的结果正确($c1180000 = -9.5$, $41180000 = +9.5$)。

除 ppt 给出的仿真之外，我还添加了一组会出现溢出的数据（相加后指数为 255），为此需要将结果设为 $+\infty$ ，即 h 7f800000，同样可以看到，结果符合预期。

还有一些细节放在实验结果分析中说明（这里可以看到，最后一组结果输出后 fin 一直为 1，这也在实验结果分析中提到。）。

四、实验结果分析

乘法器和除法器的实现原理以及实验结果在上述仿真结果中已有了较详细的分析，在这里就比较复杂的 `floatadd` 进行一些分析。



(1) 输出结果时延问题:

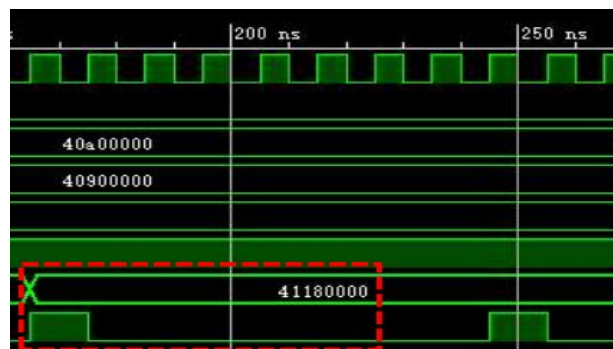
由于状态机的实现中都采用非阻塞赋值，因此在得到一组数据的时候是无法在一个时钟周期中得到输出的。

我的状态机有 000 – 110 7 个阶段，所以从获得新数据的第一个 clk 上升沿到得到答案，理论上共有 7 个时钟周期，从结果看这是合理的。

(2) 计算唯一性:

```
3'b110:begin
    fin <= 1;
    if({signa,expa,fracb} != A || {signb,expb,fracb} != B) begin
        state <= 3'b000;
    end
end
end
```

在计算结束归零时，检查了输入是否更新的语句（这也是为什么最后一组数据输出后到 `rst` 重置，`fin` 一直为 1 的原因），这可以让每次计算有唯一性，不然会出现如下 ppt 中的现象（没问题但不太舒服...）：



五、讨论与心得

做了那么多实验，这次终于搞清楚“<=”的用法了（被折磨了）。在 Verilog 中，“<=”表示非阻塞赋值。这意味着在时钟周期结束时，所有的非阻塞赋值操作都会同时执行，而不是立即生效。

我在 div32 中的 always 块中尝试了一种写法：

```
If (...) begin
    rem <= ({(rem[63:32] - div),rem[31:0]}) << 1)
    rem <= rem + 1'b1;
end
```

结果发现答案并不正确，然后就发现了“<=”的正确用法？

通过这次实验，我对基础计算（乘法、除法、浮点加减法）的原理和硬件实现有了进一步的认识和理解。特别的，在 floatadd 的实现中，我第一次尝试用复杂状态机来进行逻辑实现，获得了不错的成效。